

Cellula Technologies — NLP Internship

Project 3

Evaluator–Generator and Agentic RAG Platform

A retrieval-augmented question-answering system combining a self-critiquing two-agent core with a three-agent tool-using extension, voice input, and image OCR ingestion.

Authors: Aly Adel • Hussein Medhat
Program: Cellula Technologies NLP Internship
Document: Technical Architecture Report

September 7, 2026

Contents

1	Introduction	2
1.1	Objectives	2
1.2	Technology Stack	3
2	Part 1: The Evaluator–Generator Core	4
2.1	Architecture Overview	4
2.2	The Generator Agent	4
2.3	The Evaluator Agent	5
2.4	Memory Isolation	5
2.5	The Feedback Loop	5
2.6	Redis Caching Layer	5
2.7	Ingestion Pipeline	6
3	Part 2: The Agentic Retriever–Analyst–Answer Pipeline	7
3.1	Architecture Overview	7
3.2	The Retriever Agent	7
3.3	The Analyst Agent	8
3.3.1	Analyst Tools	8
3.4	The Answer Agent	8
4	Extended Features	9
4.1	Voice Query	9
4.2	Image Ingestion via OCR	9
5	Verified End-to-End Behaviour	10
5.1	What Was Tested	10
5.2	Known Limitations	10
	Appendix: Contribution Summary	11

Chapter 1

Introduction

This report documents the design and implementation of a retrieval-augmented generation (RAG) platform built in two stages.

Project Scope

Part 1 implements a two-agent *Evaluator–Generator* self-critique loop: a Generator answers from retrieved context, and an Evaluator scores that answer, sending it back for revision when it falls short, bounded by a maximum loop count.

Part 2 extends this into a three-agent *Retriever–Analyst–Answer* pipeline, where each agent has a narrow, well-defined responsibility and a set of dedicated tools, coordinated by an orchestrator with its own bounded feedback loop between the Analyst and the Retriever. Both pipelines share the same document ingestion layer, Redis caching strategy, and LangChain/LCEL-based orchestration pattern. The platform was further extended with **voice-based querying** and **image ingestion via OCR**.

1.1 Objectives

- Answer user questions strictly from retrieved, ingested knowledge – never from the model’s own outside knowledge.
- Self-evaluate every generated answer before returning it to the user.
- Support multiple knowledge input formats: PDF, DOCX, TXT, source code, PPTX, web pages, Wikipedia articles, and images (via OCR).
- Cache expensive operations (embeddings, retrieval, LLM calls) to control cost and latency.
- Provide transparent, cited, and honest answers – including honest refusals when evidence is insufficient.

1.2 Technology Stack

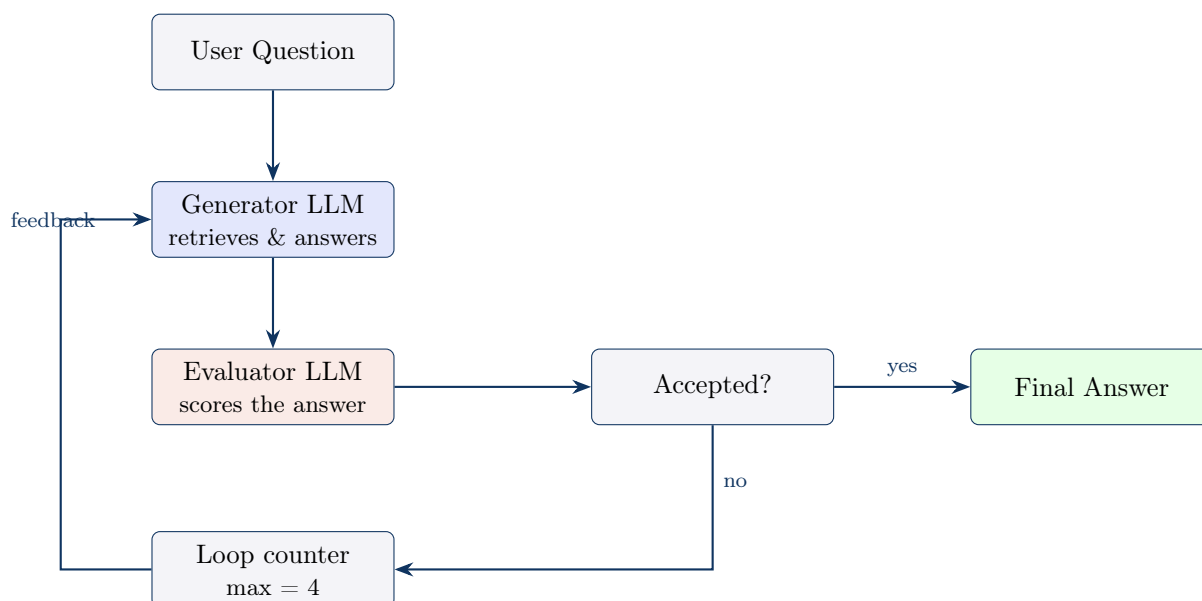
Layer	Technology
API framework	FastAPI + Uvicorn
Orchestration	LangChain / LCEL (<code>RunnableLambda</code>)
LLM provider	OpenRouter (chat, embeddings, transcription endpoints)
Vector store	Chroma
Caching	Redis
Frontend	Streamlit
OCR	Tesseract via <code>pytesseract</code>
Voice input	<code>st.audio_input</code> + OpenRouter <code>/audio/transcriptions</code>

Chapter 2

Part 1: The Evaluator–Generator Core

2.1 Architecture Overview

The core loop follows a strict grounding rule: the Generator may only use retrieved context, and must explicitly say so when the context is insufficient rather than inventing an answer. The Evaluator scores every answer against four criteria – accuracy, relevance, completeness, and grounding – and only accepts an answer once all four scores clear a 0.7 threshold.



2.2 The Generator Agent

The Generator is a plain LCEL chain: a fixed system prompt, a human template combining retrieved context and the question, a `ChatOpenAI` model call, and a `StrOutputParser`. Its own memory (`GenerateMemory`) stores conversation history, retrieved context, and previous answers, but is never shared with the Evaluator.

Listing 2.1: Generator system prompt (excerpt)

```
1 GENERATOR_SYSTEM_PROMPT = """You are the Generator in a RAG system.  
2 Answer the question using ONLY the retrieved context below -- never  
3 use outside knowledge. If the context doesn't contain enough  
4 information to answer, say so explicitly rather than guessing."""
```

2.3 The Evaluator Agent

The Evaluator receives the question, the Generator’s answer, and the retrieved context, and returns a structured `EvaluationResult` via a `PydanticOutputParser`:

Listing 2.2: Structured evaluation schema

```

1 class EvaluationResult(BaseModel):
2     accepted: bool
3     accuracy_score: float          # 0.0 - 1.0
4     relevance_score: float
5     completeness_score: float
6     grounding_score: float
7     feedback: str                  # actionable, empty if accepted

```

`accepted` is `true` only when every score is ≥ 0.7 . Otherwise, `feedback` is written back into the Generator’s next prompt so the revision addresses the specific shortfall, rather than regenerating blindly.

2.4 Memory Isolation

A mandatory architectural constraint: the Generator and Evaluator never share a memory instance. Each is backed by its own `InMemoryChatMessageHistory` wrapper, instantiated fresh per question inside the orchestrator, so there is no code path by which one agent’s memory can leak into the other’s prompt.

2.5 The Feedback Loop

Implemented as a bounded `for` loop wrapped in an LCEL `RunnableLambda`, so the whole workflow composes with the rest of the LangChain pipeline while the iteration logic itself stays an explicit, debuggable Python loop rather than a chain of `RunnableBranch` nodes.

Listing 2.3: Feedback loop, simplified

```

1 for loop_index in range(1, max_loops + 1):
2     answer, retrieved = generate_answer(question, feedback, k=k)
3     evaluation = evaluate_answer(question, answer, retrieved)
4     if evaluation.accepted:
5         return WorkflowResult(answer, accepted=True, loops_used=
6             loop_index)
7     feedback = evaluation.feedback
8 # max loops reached without acceptance
9 return WorkflowResult(answer, accepted=False, max_loops_reached=True)

```

If the maximum (4) iterations pass without acceptance, the last generated answer is still returned, tagged with `max_loops_reached=True` and a disclaimer – the system always returns something usable rather than failing outright.

2.6 Redis Caching Layer

Four categories of data are cached, each keyed by a SHA-256 hash of its inputs:

Cache	Key inputs
Embeddings	raw text
Retrieval results	query + k
Generator responses	question + context + feedback
Evaluations	question + answer + context

All Redis calls are wrapped in `try/except`; a Redis outage degrades to “always miss” rather than crashing the request.

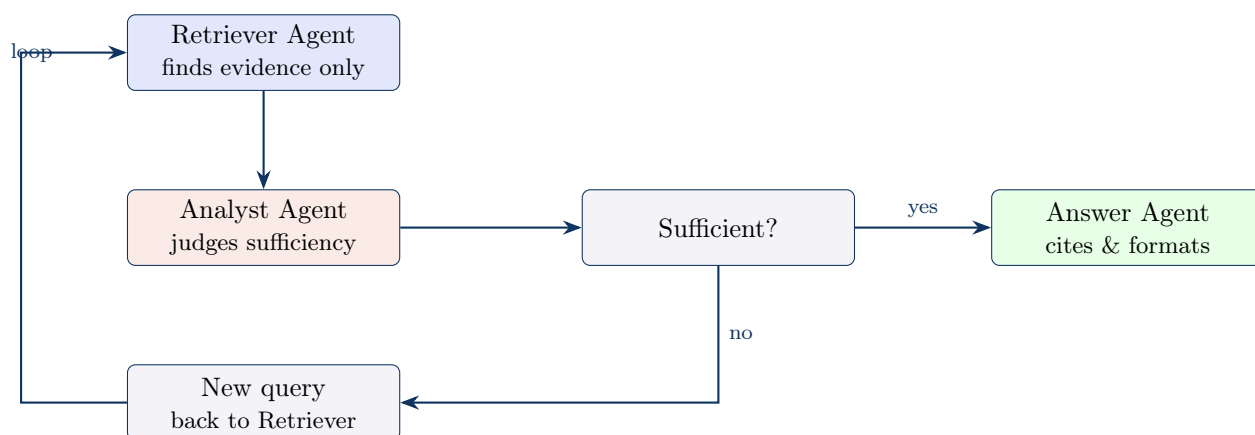
2.7 Ingestion Pipeline

A single dispatcher, `ingest_source()`, routes an uploaded file, URL, or Wikipedia query to the correct loader by `SourceType`, then chunks (`RecursiveCharacterTextSplitter`, 500 characters, 100 overlap) and embeds the result into Chroma.

Chapter 3

Part 2: The Agentic Retriever–Analyst–Answer Pipeline

3.1 Architecture Overview



Unlike Part 1’s regenerate-the-whole-answer loop, this pipeline’s feedback mechanism is *evidence-driven*: the Analyst never rewrites an answer itself – it decides whether the evidence is enough, and if not, issues a narrow follow-up query back to the Retriever.

3.2 The Retriever Agent

The Retriever’s only responsibility is: question in, evidence out – it never generates an answer. It runs a fixed six-stage pipeline.

#	Tool	Purpose
1	Query Rewriter	Resolves pronouns/follow-ups into a standalone query
2	Semantic Search	Embedding cosine-similarity search (Chroma)
3	Keyword Search	BM25 exact-term matching, catches what embeddings miss
4	Metadata Filter	Restricts search to a document/section/date
5	Reranker	LLM re-scores each candidate 0–10 against the question
6	Context Selector	Trims to a compact, deduplicated, high-score set

Design note

Search 2 and 3 run in parallel and are merged before reranking. Semantic search catches paraphrased or reworded matches; keyword search catches exact technical terms an embedding model might blur together. The reranker then corrects cases where either method's raw score is topically close but doesn't actually answer the question.

3.3 The Analyst Agent

The Analyst reasons over the evidence bundle and judges sufficiency via a structured **AnalysisAssessment**:

Listing 3.1: Analyst's structured assessment

```
1 class AnalysisAssessment(BaseModel):
2     sufficient: bool
3     missing_information: str
4     follow_up_query: str
5     requires_numeric_comparison: bool
6     labeled_values: dict[str, float]
7     key_facts: list[str]
```

3.3.1 Analyst Tools

Tool	What it does
Calculator	Safe AST-based arithmetic – never lets the LLM compute in its head
Table Extractor	Regex-first, LLM-fallback structured table extraction
Document Comparison	Cross-references claims across distinct ingested sources
Data Analysis	Statistical analysis over extracted numeric values
Evidence Request	The feedback-loop trigger: asks the Retriever for more

The sufficiency loop is bounded by the same **max_feedback_loops** setting used in Part 1, and includes an early-exit optimisation: if a follow-up query returns zero new evidence, the loop stops immediately rather than re-running an assessment that is guaranteed to reach the same conclusion.

3.4 The Answer Agent

Once the Analyst confirms sufficiency (or the loop bound is reached), the Answer Agent turns findings into the final response:

Tool	Purpose
Citation Formatter	Attaches page/section references to specific claims
Source Formatter	Lists which distinct documents were actually used
Response Formatter	Assembles the final, readable, cited answer

If the maximum analyst loops were reached without confirmed sufficiency, a disclaimer is attached and the Answer Agent is explicitly instructed to be transparent about the remaining gaps rather than papering over them.

Chapter 4

Extended Features

4.1 Voice Query

Recorded audio (via Streamlit's `st.audio_input`) is sent to a dedicated `/voice/transcribe` endpoint, which forwards it to OpenRouter's unified `/api/v1/audio/transcriptions` endpoint – using the same API key as the rest of the platform, with no additional provider integration.

Listing 4.1: Voice transcription call

```
1 response = requests.post(  
2     "https://openrouter.ai/api/v1/audio/transcriptions",  
3     headers={"Authorization": f"Bearer {api_key}"},  
4     json={"model": "openai/whisper-1",  
5         "input_audio": {"data": encoded_audio, "format": "wav"}},  
6     timeout=60,  
7 )
```

The transcribed text is then routed into the normal `/ask` pipeline exactly as if it had been typed, requiring no changes to the agentic pipeline itself.

4.2 Image Ingestion via OCR

Images are treated as another ingestible source type. Text is extracted via Tesseract OCR (`pytesseract`) and wrapped in the same `Document` object used by every other loader, so it flows through the identical `chunk/embed/store` pipeline.

Listing 4.2: OCR loader (core logic)

```
1 def extract_text_from_image(image_path, source_name="uploaded_image"):  
2     image = Image.open(image_path)  
3     text = pytesseract.image_to_string(image).strip()  
4     if not text:  
5         raise ValueError(f"No readable text found in image: {source_name}  
6         })")  
7     return [Document(content=text,  
8                     metadata={"source_name": source_name,  
9                             "source_type": "image_ocr"})]
```

Chapter 5

Verified End-to-End Behaviour

5.1 What Was Tested

- PDF, Wikipedia, and image ingestion, each producing correctly chunked, embedded, and stored content.
- A grounded, correct answer accepted by the Evaluator on the first loop, with all four scores at 1.0.
- A correct, honest refusal when retrieved context did not contain the requested fact – the Generator explicitly stated the information was unavailable rather than fabricating an answer.
- The Analyst correctly skipping Document Comparison when only a single source was present, rather than returning a hollow “no contradictions found” result.
- The full multi-loop evidence-insufficiency path, correctly returning `max_loops_reached=True` with a disclaimer after exhausting retrieval attempts on a genuinely unanswerable question.

5.2 Known Limitations

- Voice transcription requires a funded OpenRouter account – no free transcription model is currently available on the platform.
- Free-tier LLM routing (`openrouter/auto`) can occasionally return malformed or empty structured output; mitigated with a JSON-repair step and bounded retries, but not eliminated entirely.
- WAV-based audio *ingestion* (as opposed to voice *querying*) was deferred to avoid a heavy local Whisper/torch dependency footprint.

Appendix: Contribution Summary

Contributor	Scope of work
Hussein Medhat	Part 1: core agent logic – Generator, Evaluator, prompts, chunking, ingestion document model, embeddings, vector store, configuration, and the source loaders. Part 2: the Retriever and Analyst agent orchestrators, the Answer Agent and its tools (Citation Formatter, Source Formatter, Response Formatter, Evidence Request), and the agentic workflow orchestrator wiring the three agents together.
Aly Adel	Part 1: the Generator–Evaluator feedback loop orchestrator and Redis caching layer. Part 2: the Retriever agent’s tools (Query Rewriter, Semantic Search, Keyword Search, Metadata Filter, Reranker, Context Selector) and the Analyst agent’s tools (Calculator, Table Extractor, Document Comparison, Data Analysis). Both parts: ingestion pipeline dispatcher, FastAPI service layer, Streamlit UI, voice query, and image OCR ingestion.